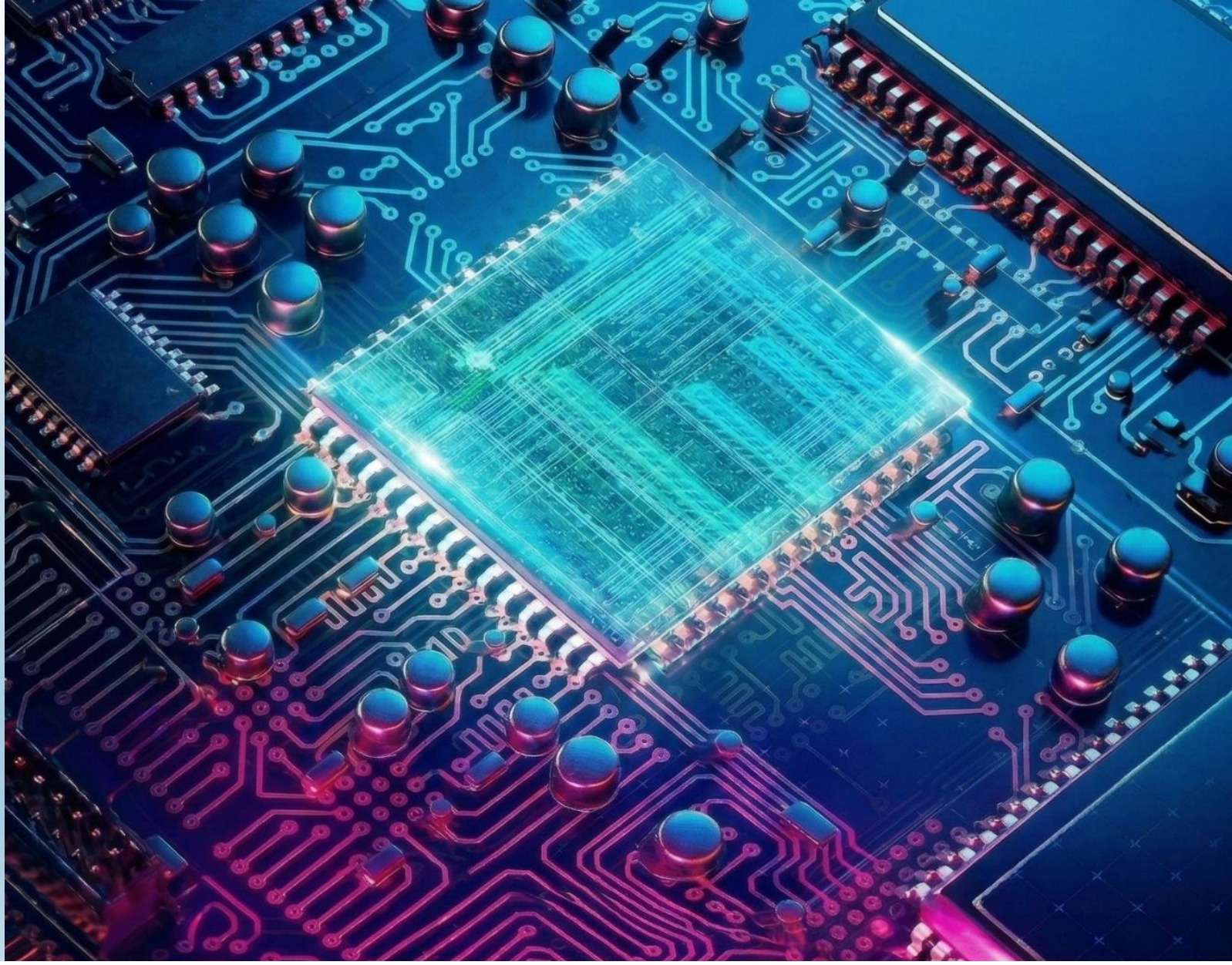


Digital Verification Assignment 2

Name: Mohamed Torki

Alexandria University, Faculty of Engineering

Electronics and Communications Department

Question 1:

Code:

Assignments > A2 > Q1 > dyn_arr_ex.sv > dyn_arr_ex

```
1   module dyn_arr_ex;
2
3   int dyn_arr1[], dyn_arr2[];
4
5   initial begin
6   dyn_arr1 = new[6];
7
8   foreach(dyn_arr1[i])
9       dyn_arr1[i] = i;
10
11  dyn_arr2 = '{9,1,8,3,4,4};
12
13  foreach(dyn_arr1[i])
14      $display("Element %0d in dyn_arr1 = %0d and its Size = %0d",i,dyn_arr1[i],dyn_arr1.size());
15
16  dyn_arr1.delete();
```

```
18  foreach(dyn_arr2[i])
19      $display("For Original Array: Element %0d in dyn_arr2 = %0d",i,dyn_arr2[i]);
20
21  dyn_arr2.reverse();
22
23  foreach(dyn_arr2[i])
24      $display("For Reversed Array: Element %0d in dyn_arr2 = %0d",i,dyn_arr2[i]);
25
26  dyn_arr2.sort();
27
28  foreach(dyn_arr2[i])
29      $display("For Sorted Array: Element %0d in dyn_arr2 = %0d",i,dyn_arr2[i]);
30
31  dyn_arr2.rsort();
32
33  foreach(dyn_arr2[i])
34      $display("For Reversly Sorted Array: Element %0d in dyn_arr2 = %0d",i,dyn_arr2[i]);
35
36  dyn_arr2.shuffle();
37
38  foreach(dyn_arr2[i])
39      $display("For Shuffled Array: Element %0d in dyn_arr2 = %0d",i,dyn_arr2[i]);
40  end
41
42  endmodule
```

Result:

```
# Element 0 in dyn_arr1 = 0 and its Size = 6
# Element 1 in dyn_arr1 = 1 and its Size = 6
# Element 2 in dyn_arr1 = 2 and its Size = 6
# Element 3 in dyn_arr1 = 3 and its Size = 6
# Element 4 in dyn_arr1 = 4 and its Size = 6
# Element 5 in dyn_arr1 = 5 and its Size = 6
# For Original Array: Element 0 in dyn_arr2 = 9
# For Original Array: Element 1 in dyn_arr2 = 1
# For Original Array: Element 2 in dyn_arr2 = 8
# For Original Array: Element 3 in dyn_arr2 = 3
# For Original Array: Element 4 in dyn_arr2 = 4
# For Original Array: Element 5 in dyn_arr2 = 4
# For Reversed Array: Element 0 in dyn_arr2 = 4
# For Reversed Array: Element 1 in dyn_arr2 = 4
# For Reversed Array: Element 2 in dyn_arr2 = 3
# For Reversed Array: Element 3 in dyn_arr2 = 8
# For Reversed Array: Element 4 in dyn_arr2 = 1
# For Reversed Array: Element 5 in dyn_arr2 = 9
# For Sorted Array: Element 0 in dyn_arr2 = 1
# For Sorted Array: Element 1 in dyn_arr2 = 3
# For Sorted Array: Element 2 in dyn_arr2 = 4
# For Sorted Array: Element 3 in dyn_arr2 = 4
# For Sorted Array: Element 4 in dyn_arr2 = 8
# For Sorted Array: Element 5 in dyn_arr2 = 9
# For Reversly Sorted Array: Element 0 in dyn_arr2 = 9
# For Reversly Sorted Array: Element 1 in dyn_arr2 = 8
# For Reversly Sorted Array: Element 2 in dyn_arr2 = 4
# For Reversly Sorted Array: Element 3 in dyn_arr2 = 4
# For Reversly Sorted Array: Element 4 in dyn_arr2 = 3
# For Reversly Sorted Array: Element 5 in dyn_arr2 = 1
# For Shuffled Array: Element 0 in dyn_arr2 = 8
# For Shuffled Array: Element 1 in dyn_arr2 = 1
# For Shuffled Array: Element 2 in dyn_arr2 = 4
# For Shuffled Array: Element 3 in dyn_arr2 = 3
# For Shuffled Array: Element 4 in dyn_arr2 = 9
# For Shuffled Array: Element 5 in dyn_arr2 = 4
```

Question 2:

Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
COUNTER_1	When the reset is asserted, the output counter value should be low	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
COUNTER_2	Randomizing Inputs with every Loop	Randomization During the simulation	-	A checker in the testbench to make sure the output is correct

Package:

Assignments > A2 > Q2 > counter_pkg.sv > { } counter_pkg > counter_class

```
1   package counter_pkg;
2
3   parameter WIDTH = 4;
4       class counter_class;
5       rand bit up_down;
6       rand logic [WIDTH - 1 : 0] data_load ;
7       rand logic rst_n, load_n, ce;
8
9       constraint c{
10          rst_n dist{0:=10, 1:=90};
11          load_n dist{0:=30, 1:=70};
12          ce dist{0:=30, 1:=70};
13          data_load inside {[0 : (2**WIDTH - 1)]};
14      }
15
16      function new();
17      this.up_down = 1;
18      this.rst_n = 1;
19      this.load_n = 1;
20      this.ce = 1;
21      this.data_load = 0;
22      endfunction
23
24      endclass
25  endpackage;
```

Testbench:

Assignments > A2 > Q2 > counter_tb.sv > counter_tb

```
1   import counter_pkg::*;
2   module counter_tb;
3
4   bit clk, rst_n, load_n, up_down, ce;
5   logic [WIDTH - 1 : 0] data_load, count_out;
6   logic max_count, zero;
7
8   integer correct_count, error_count;
9
10  counter #(.WIDTH(WIDTH)) DUT(
11      .clk(clk),
12      .rst_n(rst_n),
13      .load_n(load_n),
14      .up_down(up_down),
15      .ce(ce),
16      .data_load(data_load),
17      .count_out(count_out),
18      .max_count(max_count),
19      .zero(zero)
20  );
21
22  initial begin
23      clk = 0;
24      forever
25          #2 clk = ~clk;
26  end
27
```

```
28  logic max_count_exp, zero_exp;
29  logic [WIDTH - 1 : 0] count_out_exp;
30
31  task Golden_model;
32      if(!rst_n) begin
33          count_out_exp = 0;
34          max_count_exp = 0;
35          zero_exp = 1;
36      end
37
38      else begin
39          if(load_n) begin
40              if(ce) begin
41                  if(up_down)
42                      count_out_exp = count_out_exp + 1 ;
43                  else
44                      count_out_exp = count_out_exp - 1 ;
45              end
46              else
47                  count_out_exp = count_out_exp;
48          end
49          else
50              count_out_exp = data_load;
51      end
52
53      max_count_exp = (count_out_exp == (2**WIDTH - 1));
54      zero_exp      = (count_out_exp == 0);
55
56  endtask
```

```
58  counter_class Obj;
59
60  initial begin
61  correct_count = 0;
62  error_count = 0;
63
64  data_load = 0;
65
66  count_out_exp = 0;
67  max_count_exp = 0;
68  zero_exp = 0;
69
70  //Counter_1
71  @(negedge clk);
72  assert_reset();
73  #10;
74  assert_reset();
75  Obj = new();
76
```

```
77  //Counter_2
78  for(int i = 0; i < 20; i++) begin
79      assert(Obj.randomize());
80      rst_n = Obj.rst_n;
81      load_n = Obj.load_n;
82      up_down = Obj.up_down;
83      ce = Obj.ce;
84      data_load = Obj.data_load;
85      @(posedge clk);
86      Golden_model();
87      check_result();
88  end
89
90  $display("%t: At end of test error count is %0d and correct count = %0d",
91                  $time, error_count, correct_count);
92  $stop;
93  end
```

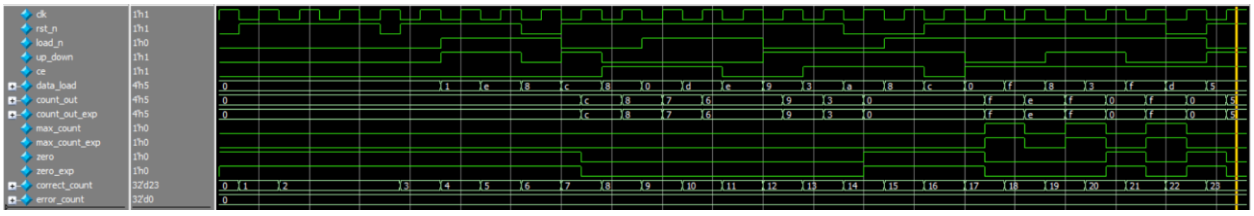
```
95  task check_result;
96
97      @(negedge clk);
98      if(max_count_exp !=  max_count || zero_exp != zero || count_out_exp != count_out)
99      begin
100         error_count ++;
101         $display("%t: Error: rst_n = %0d | load_n = %0d | up_down = %0d | ce = %0d |
102         data_load = %0d | count_out = %0d | zero = %0d | max_count = %0d | count_out_exp = %0d |
103         zero_exp = %0d | max_count_exp = %0d",$time, rst_n, load_n, up_down,
104         ce, data_load, count_out, zero, max_count, count_out_exp, zero_exp, max_count_exp);
105     end
106     else
107         correct_count++;
108 endtask
109
110 task assert_reset;
111     rst_n = 0;
112     @(posedge clk);
113     Golden_model();
114     check_result();
115
116     rst_n = 1;
117     @(posedge clk);
118     Golden_model();
119     check_result();
120 endtask
121
122 endmodule
```

Do File:

Assignments > A2 > Q2 > run.do

```
1   vlib work
2   vlog counter.v counter_tb.sv  +cover -covercells
3   vsim -voptargs=+acc work.counter_tb -cover
4   add wave *
5   coverage save counter_tb.ucdb -onexit
6   run -all
```

Waveform:



Coverage Report:

```
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        10        10         0   100.00%
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       7         7         0   100.00%
```

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         30        30         0   100.00%
```

Question 3:

Design issues:

```
11    reg signed cin_reg; //correction_1
```

```
 76            case (opcode_reg) //correction_2
```

```
97            3'h2: out <= A_reg + B_reg + cin_reg; //correction_3
```

Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
ALSU_1	Verify that all ALSU operations (OR, XOR, ADD, MULT, SHIFT, ROTATE) and invalid opcodes are tested.	Randomize opcode with higher probability for valid operations and lower probability for invalid opcodes	-	A Golden Model instantiated in the testbench is used to make sure the output is correct
ALSU_2	Design Requirement Description: Verify that bypass_A and bypass_B override all operations and directly pass A or B to output.	Randomize bypass_A and bypass_B with low probability of asserting.	-	A Golden Model instantiated in the testbench is used to make sure the output is correct
ALSU_3	Verify correct reset behavior of ALSU.	Randomize rst to be active with low probability.	-	A Golden Model instantiated in the testbench is used to make sure the output is correct
ALSU_4	Verify ADD and MULT operations with emphasis on corner cases.	Constrain A and B to take:MAXPOS, MAXNEG and ZERO more often	-	A Golden Model instantiated in the testbench is used to make sure the output is correct
ALSU_5	Verify OR and XOR operations with emphasis on corner cases.	Constrain A and B to take:MAXPOS, MAXNEG and ZERO more often	-	A Golden Model instantiated in the testbench is used to make sure the output is correct
ALSU_6	Verify SHIFT and ROTATE operations in both directions.	Randomize direction with slightly higher probability for left direction.	-	A Golden Model instantiated in the testbench is used to make sure the output is correct
ALSU_7	When the reset is asserted, the outputs should be low	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct using A Golden Model
ALSU_8	Looping and exhaustivly testing randmoized values	Directed during simulation through Randomization	-	A checker in the testbench to make sure the output is correct using A Golden Model

Golden Model:

Assignments > A2 > Q3 > ALSU_GM.v > ALSU_GM

```v
module ALSU_GM #(
    parameter INPUT_PRIORITY = "A",
    parameter FULL_ADDER = "ON")
    (
    input signed [2:0] A, B,
    input [2:0] opcode,
    input clk, rst, cin, serial_in, direction,
    input red_op_A, red_op_B, bypass_A, bypass_B,

    output reg [15:0] leds,
    output reg signed [5:0] out
);

// Signals Sampling
reg signed [2:0] A_reg, B_reg;
reg [2:0] opcode_reg;
reg signed cin_reg;
reg serial_in_reg, direction_reg;
reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg;

always@(posedge clk or posedge rst) begin
    if(rst) begin
        A_reg <= 0;
        B_reg <= 0;
        opcode_reg <= 0;
        cin_reg <= 0;
        serial_in_reg <= 0;
        direction_reg <= 0;
        red_op_A_reg <= 0;
        red_op_B_reg <= 0;
        bypass_A_reg <= 0;
        bypass_B_reg <= 0;
    end
    else begin
        A_reg <= A;
        B_reg <= B;
        opcode_reg <= opcode;
        cin_reg <= cin;
        serial_in_reg <= serial_in;
        direction_reg <= direction;
        red_op_A_reg <= red_op_A;
        red_op_B_reg <= red_op_B;
        bypass_A_reg <= bypass_A;
        bypass_B_reg <= bypass_B;
    end
 end

//Invalid Cases
assign invalid = ((red_op_A_reg || red_op_B_reg) &&
                  !(opcode_reg == 3'b000 || opcode_reg == 3'b001)) ||
                  (opcode_reg == 3'b110 || opcode_reg == 3'b111);

always @(posedge clk or posedge rst) begin
  if(rst) begin
     leds <= 0;
  end else begin
      if (invalid)
        leds <= ~leds;
      else
        leds <= 0;
  end
end

```

```v
//ALSU Logic
always@(posedge clk or posedge rst) begin
    if(rst)
        out <= 0;
    else begin
        if (bypass_A_reg && bypass_B_reg)
            out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;

        else if (bypass_A_reg)
            out <= A_reg;

        else if (bypass_B_reg)
            out <= B_reg;

        else if (invalid)
            out <= 0;
        else begin
            case(opcode_reg)
            3'b000: begin
                if(red_op_A_reg && red_op_B_reg)
                    out <= (INPUT_PRIORITY == "A")? |A_reg : |B_reg;
                else if (red_op_A_reg)
                    out <= |A_reg;
                else if (red_op_B_reg)
                    out <= |B_reg;
                else
                    out <= A_reg | B_reg;
                end
            3'b001: begin
                if(red_op_A_reg && red_op_B_reg)
                    out <= (INPUT_PRIORITY == "A")? ^A_reg : ^B_reg;
                else if (red_op_A_reg)
                    out <= ^A_reg;
                else if (red_op_B_reg)
                    out <= ^B_reg;
                else
                    out <= A_reg ^ B_reg;
                end
            3'b010: begin
                if(FULL_ADDER == "ON")
                    out <= A_reg + B_reg + cin_reg;
                else
                    out <= A_reg + B_reg;
                end
            3'b011:
                out <= A_reg * B_reg;
            3'b100: begin
                if (direction_reg)
                out <= {out[4:0], serial_in_reg};
                else
                out <= {serial_in_reg, out[5:1]};
                end
              3'b101: begin
                  if (direction_reg)
                  out <= {out[4:0], out[5]};
                  else
                  out <= {out[0], out[5:1]};
              end
              endcase
          end
      end
end

endmodule
```

Package:

Assignments > A2 > Q3 > ALSU_pkg.sv > { } ALSU_pkg > ALSU_class

```
package ALSU_pkg;
typedef enum logic [2:0] {OR, XOR, ADD, MULT, SHIFT, ROTATE, INVALID_6, INVALID_7} opcode_e;

parameter INPUT_PRIORITY = "A";
parameter FULL_ADDER = "ON";

localparam MAXPOS = 3;
localparam MAXNEG = -4;
localparam ZERO = 0;

class ALSU_class;
    rand logic rst, cin, serial_in, direction, red_op_A, red_op_B, bypass_A, bypass_B;
    rand opcode_e opcode;
    rand logic signed [2:0] A, B;

    constraint c {
        //ALSU_1
        opcode dist {OR := 80, XOR := 80, ADD := 80, MULT := 80, SHIFT := 80,
                     ROTATE := 80, INVALID_6 := 20, INVALID_7 := 20};
        //ALSU_2
        bypass_A dist {0:= 90, 1:=10};
        bypass_B dist {0:= 90, 1:=10};

        //ALSU_3
        rst dist {0:= 90, 1:= 10};

        //ALSU_4
        if(opcode inside {ADD, MULT}) {
            A dist {MAXPOS := 80, MAXNEG := 80, ZERO := 80, -3 := 20,
                    -2 := 20, -1 := 20, 1  := 20, 2  := 20};

            B dist {MAXPOS := 80, MAXNEG := 80, ZERO := 80, -3 := 20,
                    -2 := 20, -1 := 20, 1  := 20, 2  := 20};
        }
        //ALSU_5
        else if (opcode inside {OR, XOR}) {
            red_op_A dist {0:=50, 1:= 50};
            red_op_B dist {0:=50, 1:= 50};
            if(red_op_A) {
                A dist {
                    3'b001 := 70,
                    3'b010 := 70,
                    3'b100 := 70,

                    3'b011 := 20,
                    3'b101 := 20,
                    3'b110 := 20,

                    3'b000 := 10,
                    3'b111 := 10
                };
                B == 0;
            }
            else if (red_op_B) {
                B dist {
                    3'b001 := 70,
                    3'b010 := 70,
                    3'b100 := 70,

                    3'b011 := 20,
                    3'b101 := 20,
                    3'b110 := 20,

                    3'b000 := 10,
                    3'b111 := 10
                };
                A == 0;
            }

        }
        //ALSU_6
        else
            direction dist {0:=50, 1:= 80};
    }
```

```
    function new();
        this.rst = 0;
        this.cin = 0;
        this.serial_in = 0;
        this.direction = 0;
        this.red_op_A = 0;
        this.red_op_B = 0;
        this.bypass_A = 0;
        this.bypass_B = 0;
        this.opcode = OR;
        this.A = 0;
        this.B = 0;
        endfunction
endclass

endpackage
```

Testbench:

Assignments > A2 > Q3 > ALSU_tb.sv > ALSU_tb

```
1   import ALSU_pkg ::*;
2   module ALSU_tb;
3
4   bit clk, rst;
5   logic cin, serial_in, direction, red_op_A, red_op_B, bypass_A, bypass_B;
6   logic signed [2:0] A, B;
7   logic [2:0] opcode;
8
9   logic [15:0] leds;
10  logic signed [5:0] out;
11
12  logic [15:0] leds_exp;
13  logic signed [5:0] out_exp;
14
15  localparam FULL_ADDER = "ON";
16  localparam INPUT_PRIORITY = "A";
17
18  integer correct_count, error_count;
19
20
21  initial begin
22      clk = 0;
23      forever
24          #2 clk = ~clk;
25  end
26
27  ALSU #(.FULL_ADDER(FULL_ADDER), .INPUT_PRIORITY(INPUT_PRIORITY)) DUT(.*);
29  ALSU_GM #(.FULL_ADDER(FULL_ADDER), .INPUT_PRIORITY(INPUT_PRIORITY)) Golden_Model(
30      .clk(clk),
31      .rst(rst),
32      .cin(cin),
33      .serial_in(serial_in),
34      .direction(direction),
35      .red_op_A(red_op_A),
36      .red_op_B(red_op_B),
37      .bypass_A(bypass_A),
38      .bypass_B(bypass_B),
39      .A(A),
40      .B(B),
41      .opcode(opcode),
42
43      .out(out_exp),
44      .leds(leds_exp)
45  );
46
47  ALSU_class Obj;
48
```

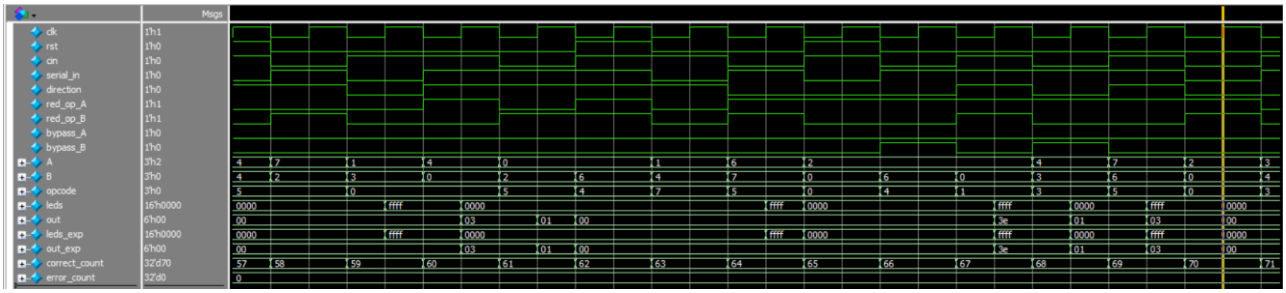
```
49  initial begin
50
51  correct_count = 0;
52  error_count = 0;
53
54  //ALSU_7
55  @(negedge clk);
56  assert_reset();
57  assert_reset();
58
59  Obj = new();
60
61  //ALSU_8
62  for(int i = 0; i < 300; i++) begin
63      assert(Obj.randomize());
64      opcode = Obj.opcode;
65      A = Obj.A;
66      B = Obj.B;
67      rst =  Obj.rst;
68      cin = Obj.cin;
69      serial_in = Obj.serial_in;
70      direction = Obj.direction;
71      red_op_A = Obj.red_op_A;
72      red_op_B = Obj.red_op_B;
73      bypass_A = Obj.bypass_A;
74      bypass_B =Obj.bypass_B;
75
76      check_result();
77  end
```

```
79  $display("%t: At end of test error count is %0d and correct count = %0d",
80                  $time, error_count, correct_count);
81  $stop;
82  end
83
84  task check_result;
85      @(negedge clk);
86      if(leds != leds_exp  || out != out_exp)
87      begin
88          error_count ++;
89          $display("%t: Error: OPCODE = %0d | leds = %0d | out = %0d | leds_exp = %0d |
90                      out_exp = %0d", $time,opcode, leds, out, leds_exp, out_exp);
91      end
92      else
93          correct_count++;
94  endtask
95
96  task assert_reset;
97      rst = 1;
98      check_result();
99      rst = 0;
100     check_result();
101 endtask
102
103 endmodule
```

Do File:

```
Assignments > A2 > Q3 >  run.do
1   vlib work
2   vlog ALSU.v ALSU_tb.sv  +cover -covercells
3   vsim -voptargs=+acc work.ALSU_tb -cover
4   add wave *
5   coverage save ALSU_tb.ucdb -onexit
6   run -all
```

Waveform:



Coverage Report:

```
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        32        32         0   100.00%
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      48        48         0   100.00%
```

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                        118       118         0   100.00%
```